

Desarrollo de Aplicaciones Web y Patrones Semana 9



Introducción a Spring Security

Uno de los elementos esenciales hoy en día, cuando casi todas las aplicaciones web dependen de datos sensibles, está relacionado con la seguridad. Spring Security es una solución que fue diseñada específicamente para abordar esta necesidad. Un framework desarrollado para desarrolladores de Java, especialmente para aquellos que desarrollan programas basados en Java utilizando Spring, para usar como protección para sus aplicaciones contra el acceso no autorizado y ataques comunes que podrían comprometer la información del usuario.

Lo que es intrigante acerca de Spring Security es que no se limita a bloquear o permitir el acceso, sino que puede ser muy versátil. Configuramos permisos en una variedad de niveles, control de sesiones, autenticación con usuarios y contraseñas o podemos integrar sistemas modernos como Google, Facebook o tokens de seguridad. Esto ha significado que la misma configuración que hace que la seguridad funcione para grandes corporaciones puede tener algo para que los estudiantes y pequeñas startups realmente construyan proyectos más seguros.

Además, Spring Security evoluciona constantemente para abordar nuevas amenazas digitales. La gran bendición que ofrece es que no pierde tiempo, simplifica la seguridad para que nuestro único enfoque sea en lo que realmente importa: la creación de aplicaciones amigables y confiables para el usuario. Desarrollar conocimientos en el dominio no solo perfecciona las habilidades técnicas, sino que también abre el camino a oportunidades en un mercado laboral donde la seguridad está ganando importancia creciente.

Esquema de roles en Spring Security

En Spring Security, la seguridad se gestiona principalmente a través de roles. Estos roles permiten definir qué acciones puede realizar cada tipo de usuario dentro de la aplicación. Un rol no es más que una etiqueta asociada a un conjunto de permisos, y dependiendo del rol asignado, el sistema decide si un usuario puede acceder o no a determinados recursos. En este modelo proponemos tres roles: ADMIN, VENDEDOR y USUARIO.

ADMIN (Administrador)	VENDEDOR	USUARIO (Cliente final)
• Es el rol con acceso total. • Puede crear, modificar y eliminar información en el sistema. • También tiene la capacidad de gestionar usuarios, revisar estadísticas y configurar cualquier aspecto de la aplicación. • En Spring Security, este rol suele representarse con permisos globales (ROLE_ADMIN), que abarcan todos los endpoints y operaciones	• Tiene un rol intermedio. • Puede visualizar la información normal del sistema, como catálogos de productos, reportes básicos y el estado de ventas, pero no tiene permisos para modificar, eliminar o crear datos. • Esto significa que su acceso está restringido a la consulta de información, evitando que afecte procesos críticos. • En términos de configuración, su acceso podría limitarse a rutas específicas de solo lectura	• Es el rol más restringido. • Solo puede gestionar el pago dentro del carrito de compras, es decir, acciones relacionadas con la compra de productos, confirmación de pedidos y seguimiento de sus transacciones. • No tiene acceso a inventarios, reportes ni configuraciones. • En Spring Security, este rol se configura con permisos mínimos, orientados únicamente a las funcionalidades del carrito (ROLE_USER).

Este modelo asegura un control adecuado de accesos, protege los datos críticos y permite que cada usuario tenga exactamente las funcionalidades que necesita, ni más ni menos.

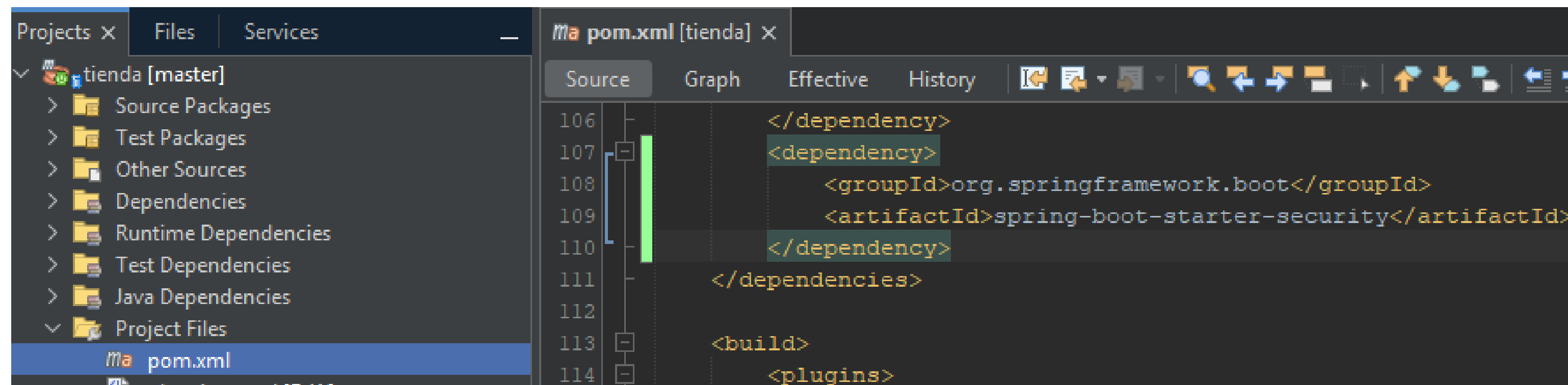
Perfiles de usuarios

	usuario	Perfil	Nivel de acceso en el sistema
	Juan	admin	Acceso a todo el sistema.
	Rebeca	vendedor	Página principal y listado de toda la información (no modifica, no elimina ni agrega).
	Pedro	usuario	Página principal y carrito de compras.

Primera dependencia

Cuando usamos Spring Boot para un proyecto web, basta con incluir la dependencia de Spring Security en el archivo pom.xml. Esta dependencia trae todo lo necesario para manejar autenticación y autorización. Al agregarla, automáticamente la aplicación queda protegida con un login básico por defecto.

Aquí un ejemplo de cómo se coloca:



```
106 </dependency>
107 <dependency>
108     <groupId>org.springframework.boot</groupId>
109     <artifactId>spring-boot-starter-security</artifactId>
110 </dependency>
111 </dependencies>
112
113 <build>
114     <plugins>
```

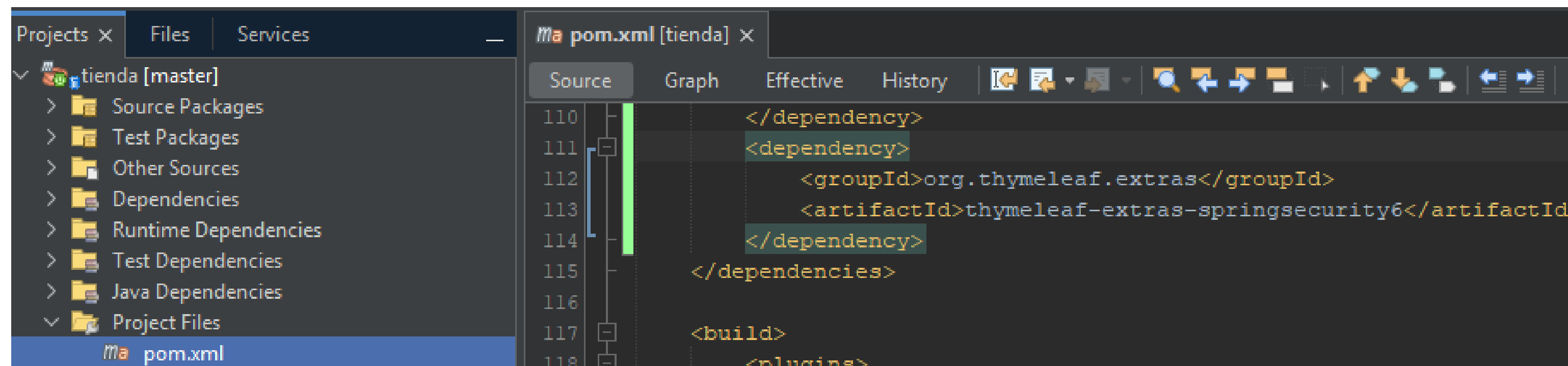
Con solo esto, Spring Security se integra en tu proyecto y ya puedes empezar a personalizar roles, accesos y reglas de seguridad.

Segunda dependencia

Sirve para integrar Spring Security con Thymeleaf, que es el motor de plantillas más usado en aplicaciones web con Spring Boot.

¿Qué permite hacer?, Con esta dependencia, en las páginas Thymeleaf puede:

- Mostrar u ocultar botones,
- enlaces o secciones según el rol del usuario.
- Validar si un usuario está autenticado o no.
- Personalizar la interfaz de acuerdo con permisos específicos.

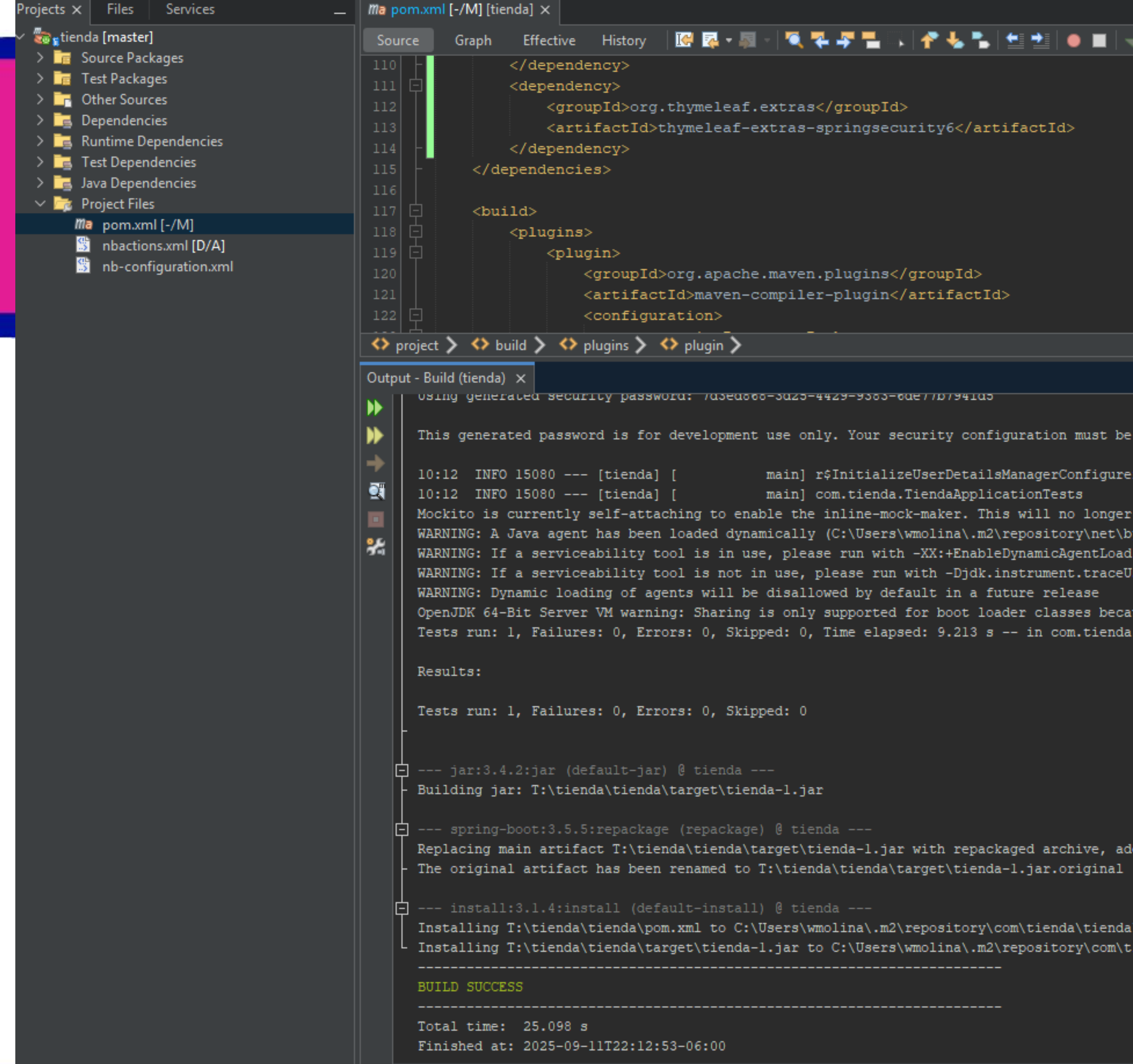


En resumen: sin esta dependencia, Thymeleaf no entendería las expresiones de seguridad de Spring Security. Con ella, puedes hacer que tu aplicación sea más dinámica y segura desde la interfaz..

Clean & Build

Antes de avanzar se hace Clean & Build (Marticillo y Escoba) para descargar las dependencias y que se puedan utilizar en lo que sigue.

Si no se realiza esto, no se reconocen las Clases y Métodos de Spring Security.



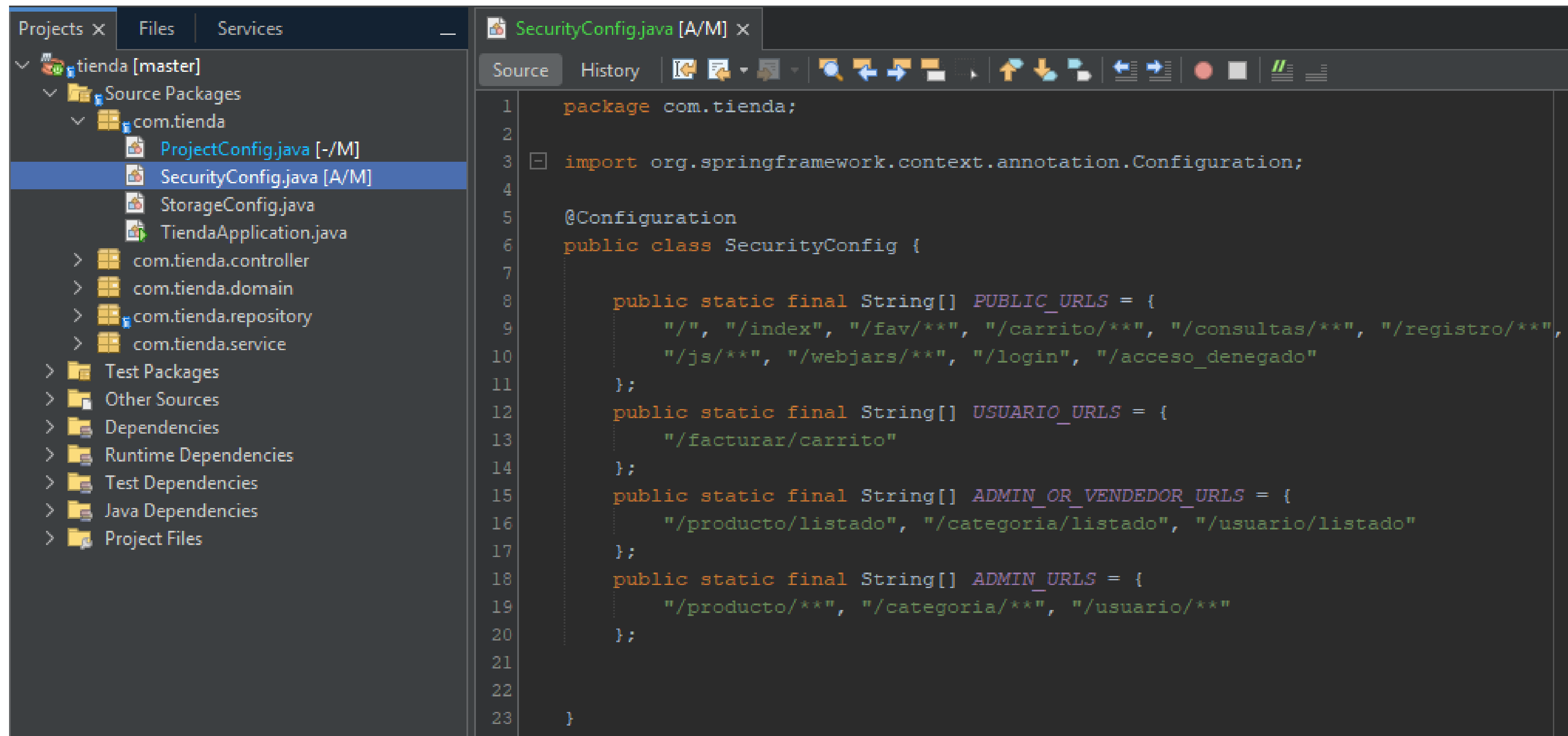
El método securityFilterChain

- Definición de Reglas de Acceso. El método securityFilterChain utiliza el patrón de encadenamiento para aplicar las reglas de forma secuencial.
 - requestMatchers(PUBLIC_URLS).permitAll()**: Permite el acceso a todas las URLs públicas sin necesidad de autenticación.
 - requestMatchers(...).hasRole(...)**: Requiere que el usuario tenga un rol específico para acceder.
 - requestMatchers(...).hasAnyRole(...)**: Permite el acceso si el usuario tiene cualquiera de los roles listados.
 - anyRequest().authenticated()**: Esta es la regla por defecto. Cualquier otra URL no listada arriba requiere que el usuario esté autenticado para acceder.
- Manejo de Autenticación y Sesiones. Se configuran los comportamientos del sistema de seguridad
 - formLogin()**: Habilita el inicio de sesión con un formulario y especifica la página de login (/login).
 - logout()**: Configura el cierre de sesión, borrando la sesión y las cookies.
 - exceptionHandling()**: Redirige a una página de "Acceso Denegado" (/acceso_denegado) si un usuario intenta acceder a una URL para la que no tiene permisos.
 - sessionManagement()**: Limita la cantidad de sesiones activas por usuario para mayor seguridad.

```
SecurityConfig.java [A/M] x
Source History
25 @Bean
26 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
27 {
28     http.authorizeHttpRequests(request -> request
29         .requestMatchers(PUBLIC_URLS).permitAll()
30         .requestMatchers(USUARIO_URLS).hasRole("USUARIO")
31         .requestMatchers(ADMIN_OR_VENDEDOR_URLS).hasAnyRole("ADMIN", "VENDEDOR")
32         .requestMatchers(ADMIN_URLS).hasRole("ADMIN")
33         .anyRequest().authenticated()
34     ).formLogin(form -> form // Configuración de formulario de login
35         .loginPage("/login")
36         .loginProcessingUrl("/login")
37         .defaultSuccessUrl("/", true)
38         .failureUrl("/login?error=true")
39         .permitAll()
40     ).logout(logout -> logout // Configuración de logout
41         .logoutUrl("/logout")
42         .logoutSuccessUrl("/login?logout=true")
43         .invalidateHttpSession(true)
44         .deleteCookies("JSESSIONID")
45         .permitAll()
46     ).exceptionHandling(exceptions -> exceptions // Manejo de excepciones
47         .accessDeniedPage("/acceso_denegado")
48     ).sessionManagement(session -> session // Configuración de sesiones
49         .maximumSessions(1)
50         .maxSessionsPreventsLogin(false)
51     );
52     return http.build();
53 }
```


Definiendo las rutas

- Este código define la cadena de filtros de seguridad que protege tu aplicación web. Controla qué usuarios pueden acceder a qué URLs basándose en sus roles.
- Agrupación de URLs por Tipo de Acceso. Se definen constantes que agrupan las URLs para una gestión clara y centralizada.
 - **PUBLIC_URLS**: Rutas de acceso libre. Cualquier persona (autenticada o no) puede entrar.
 - **USUARIO_URLS**: Rutas para usuarios con el rol USUARIO.
 - **ADMIN_OR_VENDEDOR_URLS**: Rutas que pueden ser accedidas por los roles ADMIN y VENDEDOR.
 - **ADMIN_URLS**: Rutas exclusivas para el rol ADMIN.



```
1 package com.tienda;
2
3 import org.springframework.context.annotation.Configuration;
4
5 @Configuration
6 public class SecurityConfig {
7
8     public static final String[] PUBLIC_URLS = {
9         "/", "/index", "/fav/**", "/carrito/**", "/consultas/**", "/registro/**",
10        "/js/**", "/webjars/**", "/login", "/acceso_denegado"
11    };
12
13     public static final String[] USUARIO_URLS = {
14         "/facturar/carrito"
15     };
16
17     public static final String[] ADMIN_OR_VENDEDOR_URLS = {
18         "/producto/listado", "/categoria/listado", "/usuario/listado"
19     };
20
21     public static final String[] ADMIN_URLS = {
22         "/producto/**", "/categoria/**", "/usuario/**"
23     };
24 }
```

Configuración de Usuarios de Spring Security (Fase de Desarrollo)

1. PasswordEncoder

Este bean es fundamental para la seguridad. Define el método de encriptación de contraseñas que la aplicación utilizará.

BCrypt: Se utiliza BCryptPasswordEncoder, que es el estándar de la industria. Encripta las contraseñas para que nunca se almacenen en texto plano, protegiendo a los usuarios.

2. UserDetailsService (Gestión de Usuarios)

Este bean es el encargado de gestionar los datos de los usuarios (nombre, contraseña y roles).

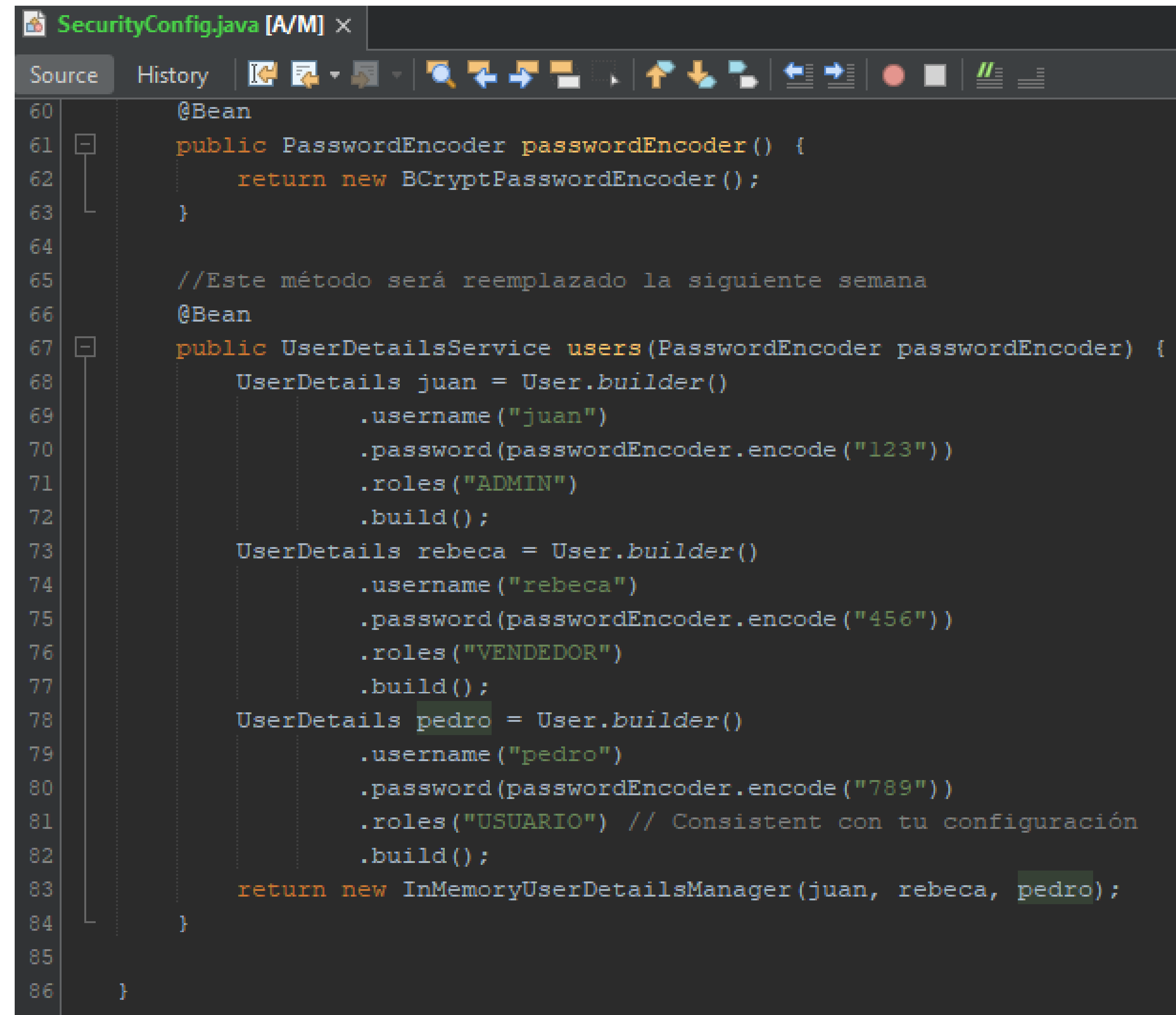
En Memoria (InMemoryUserDetailsManager): En esta etapa, los usuarios (juan, rebeca, pedro) y sus roles se crean directamente en el código y se almacenan únicamente en la memoria de la aplicación.

3. Consideraciones Importantes

Esta configuración es ideal para la fase de desarrollo y pruebas rápidas, ya que no se necesita una base de datos.

Limitación: Los usuarios se pierden cada vez que la aplicación se reinicia.

Siguiente Paso: Para un entorno de producción, este método será reemplazado por una solución persistente que se conecte a una base de datos.

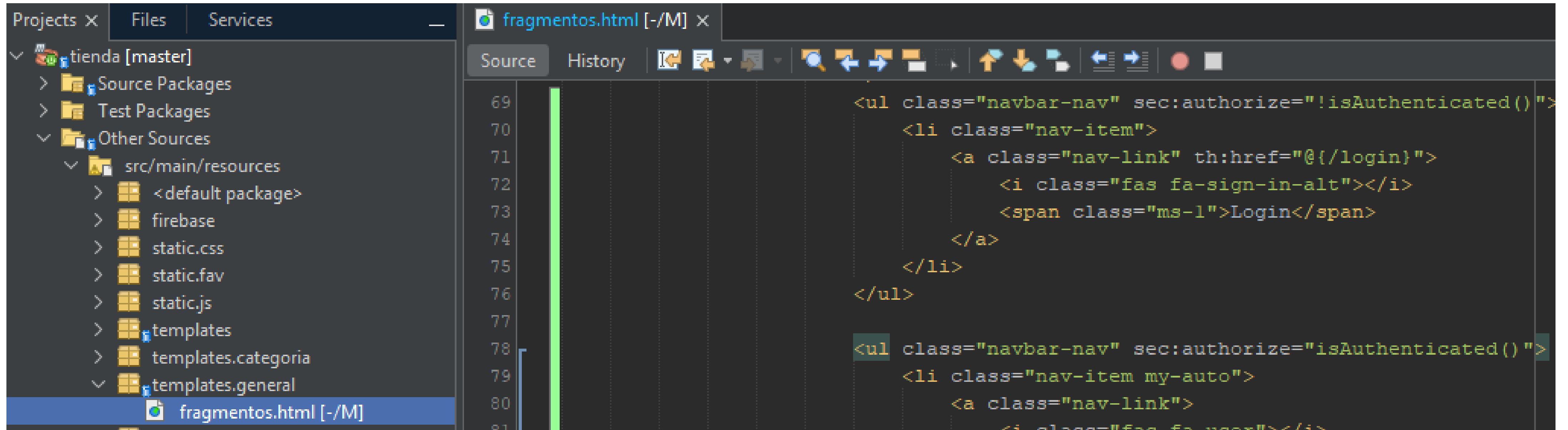


```
SecurityConfig.java [A/M] x
Source History
60 @Bean
61 public PasswordEncoder passwordEncoder() {
62     return new BCryptPasswordEncoder();
63 }
64
65 //Este método será reemplazado la siguiente semana
66 @Bean
67 public UserDetailsService users(PasswordEncoder passwordEncoder) {
68     UserDetails juan = User.builder()
69         .username("juan")
70         .password(passwordEncoder.encode("123"))
71         .roles("ADMIN")
72         .build();
73     UserDetails rebeca = User.builder()
74         .username("rebeca")
75         .password(passwordEncoder.encode("456"))
76         .roles("VENDEDOR")
77         .build();
78     UserDetails pedro = User.builder()
79         .username("pedro")
80         .password(passwordEncoder.encode("789"))
81         .roles("USUARIO") // Consistent con tu configuración
82         .build();
83     return new InMemoryUserDetailsManager(juan, rebeca, pedro);
84 }
85
86 }
```


login.html

```
login.html [A/M] x
Source History
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
  <head th:replace="~{general/fragmentos :: head}"/>
  <body>
    <header th:replace="~{general/fragmentos :: header}"/>
    <div class="row my-5">
      <div class="col-md-4 offset-md-4">
        <div class="card">
          <div class="card-header bg-success text-white text-center">
            <h2>[[#{login.login}]]</h2>
          </div>
          <div class="card-body">
            <form method="POST" th:action="@{/login}">
              <!-- Formulario y campos -->
              <div class="row my-3">
                <label for="username" class="col-md-1 my-auto"><i class="fas fa-user-lock"></i></label>
                <div class="col-md-11 my-auto">
                  <input class="form-control" id="username" type="text" name="username" th:placeholder="#{login.username}"/>
                </div>
              </div>
              <div class="row my-3">
                <label for="password" class="col-md-1 my-auto"><i class="fas fa-key"></i></label>
                <div class="col-md-11 my-auto">
                  <input class="form-control" id="password" type="password" name="password" th:placeholder="#{login.password}"/>
                </div>
              </div>
              <!-- Botones y enlaces -->
              <div class="d-flex justify-content-around mt-4">
                <!-- Enlace de registro (con estilo de botón) -->
                <a th:href="@{/registro/nuevo}" class="btn btn-primary"><i class="fas fa-user-plus"></i> [[#{accion.registrar}]]</a>
                <!-- Enlace para recordar contraseña -->
                <a th:href="@{/registro/recordar}" class="btn btn-info"><i class="fas fa-envelope"></i> [[#{accion.recordar}]]</a>
                <!-- Botón para enviar el formulario -->
                <button class="btn btn-success" type="submit"><i class="fas fa-sign-in-alt"></i> [[#{login.login}]]</button>
              </div>
            </form>
          </div>
        </div>
      </div>
      <!-- Manejo de Errores -->
      <div th:if="${param.error != null}" class="alert alert-danger mt-3 text-center" role="alert">
        [[#{error.login}]]
      </div>
    </div>
  </div>
  <footer th:replace="~{general/fragmentos :: footer}"/>
</body>
</html>
```

Incluyendo un icono para hacer login en general \fragmentos



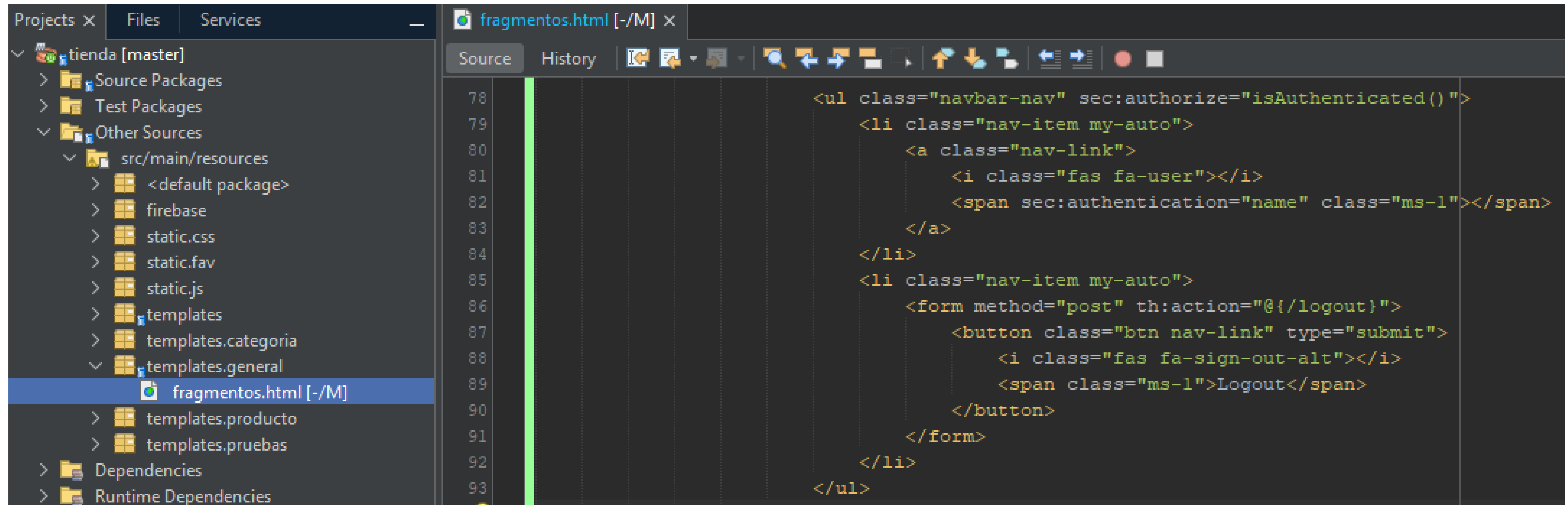
The screenshot shows an IDE with a project named 'tienda [master]'. The file explorer on the left shows the following structure:

- tienda [master]
 - Source Packages
 - Test Packages
 - Other Sources
 - src/main/resources
 - <default package>
 - firebase
 - static.css
 - static.fav
 - static.js
 - templates
 - templates.categoria
 - templates.general

The main editor displays the code for 'fragmentos.html' with line numbers 69 to 81. The code is as follows:

```
69      <ul class="navbar-nav" sec:authorize="!isAuthenticated()">
70          <li class="nav-item">
71              <a class="nav-link" th:href="@{/login}">
72                  <i class="fas fa-sign-in-alt"></i>
73                  <span class="ms-1">Login</span>
74              </a>
75          </li>
76      </ul>
77
78      <ul class="navbar-nav" sec:authorize="isAuthenticated()">
79          <li class="nav-item my-auto">
80              <a class="nav-link">
81                  <i class="fas fa-user"></i>
```

Incluyendo ícono para logout en general\fragmentos



The screenshot shows an IDE with the following structure:

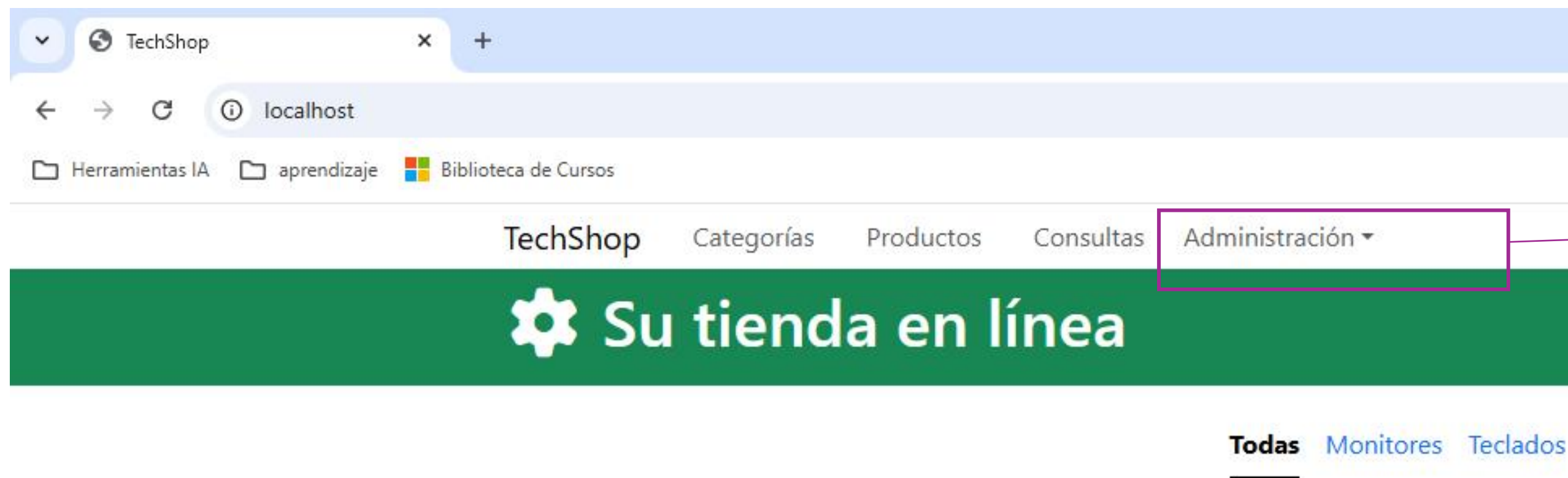
- Projects: tienda [master]
 - Source Packages
 - Test Packages
 - Other Sources
 - src/main/resources
 - <default package>
 - firebase
 - static.css
 - static.fav
 - static.js
 - templates
 - templates.categoria
 - templates.general
 - fragmentos.html [-/M] (selected)
 - templates.producto
 - templates.pruebas
 - Dependencies
 - Runtime Dependencies

The code in 'fragmentos.html' is as follows:

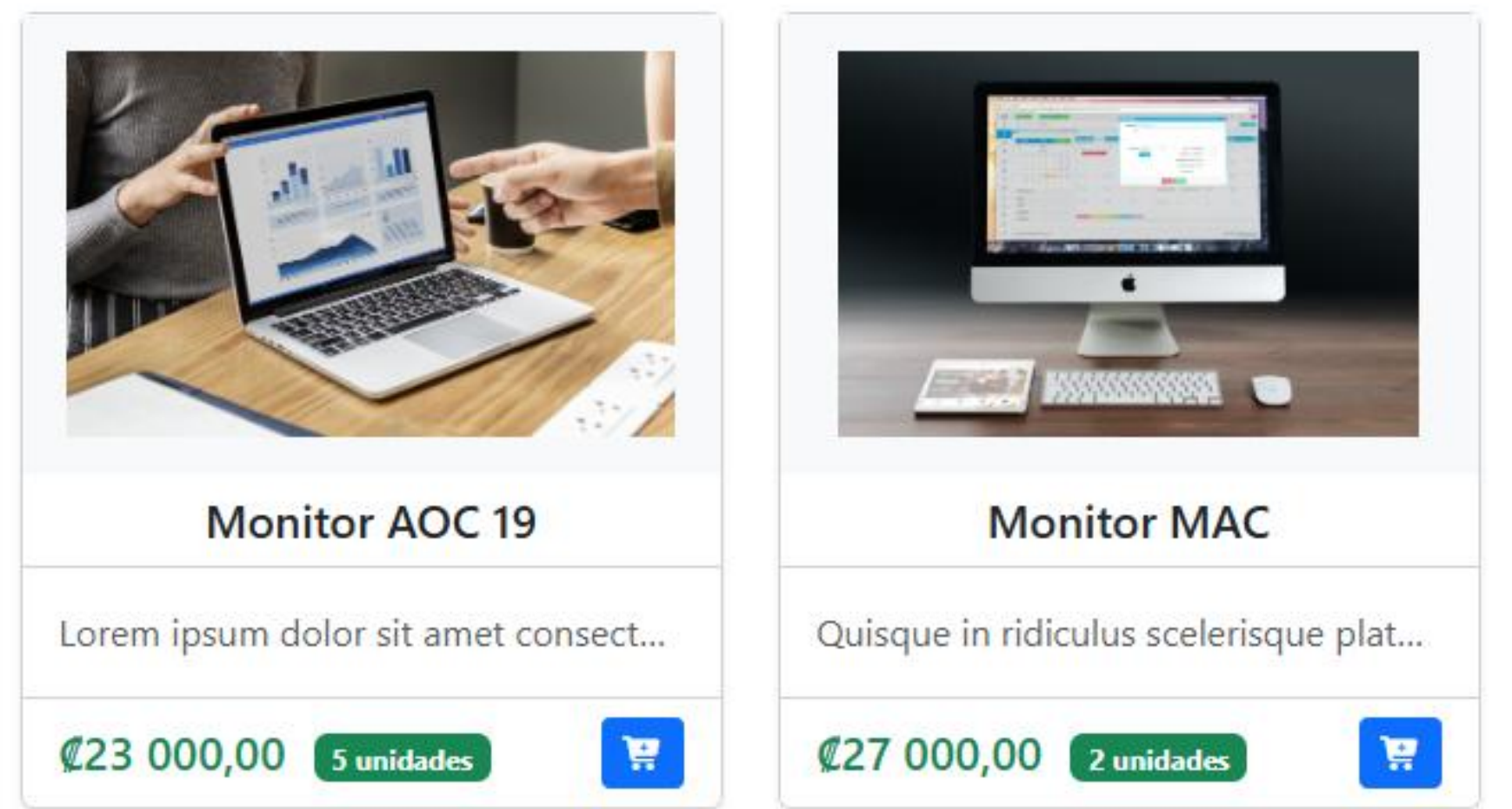
```
78 <ul class="navbar-nav" sec:authorize="isAuthenticated()">
79
80   <li class="nav-item my-auto">
81     <a class="nav-link">
82       <i class="fas fa-user"></i>
83       <span sec:authentication="name" class="ms-1"></span>
84     </a>
85   </li>
86   <li class="nav-item my-auto">
87     <form method="post" th:action="@{/logout}">
88       <button class="btn nav-link" type="submit">
89         <i class="fas fa-sign-out-alt"></i>
90         <span class="ms-1">Logout</span>
91       </button>
92     </form>
93   </li>
</ul>
```


Restringiendo el menú con `sec:authorize`

La sentencia `sec:authorize` permite generar un código HTML si el resultado de su evaluación es verdadero, como por ejemplo:



`sec:authorize="hasRole('ROLE_ADMIN')"`



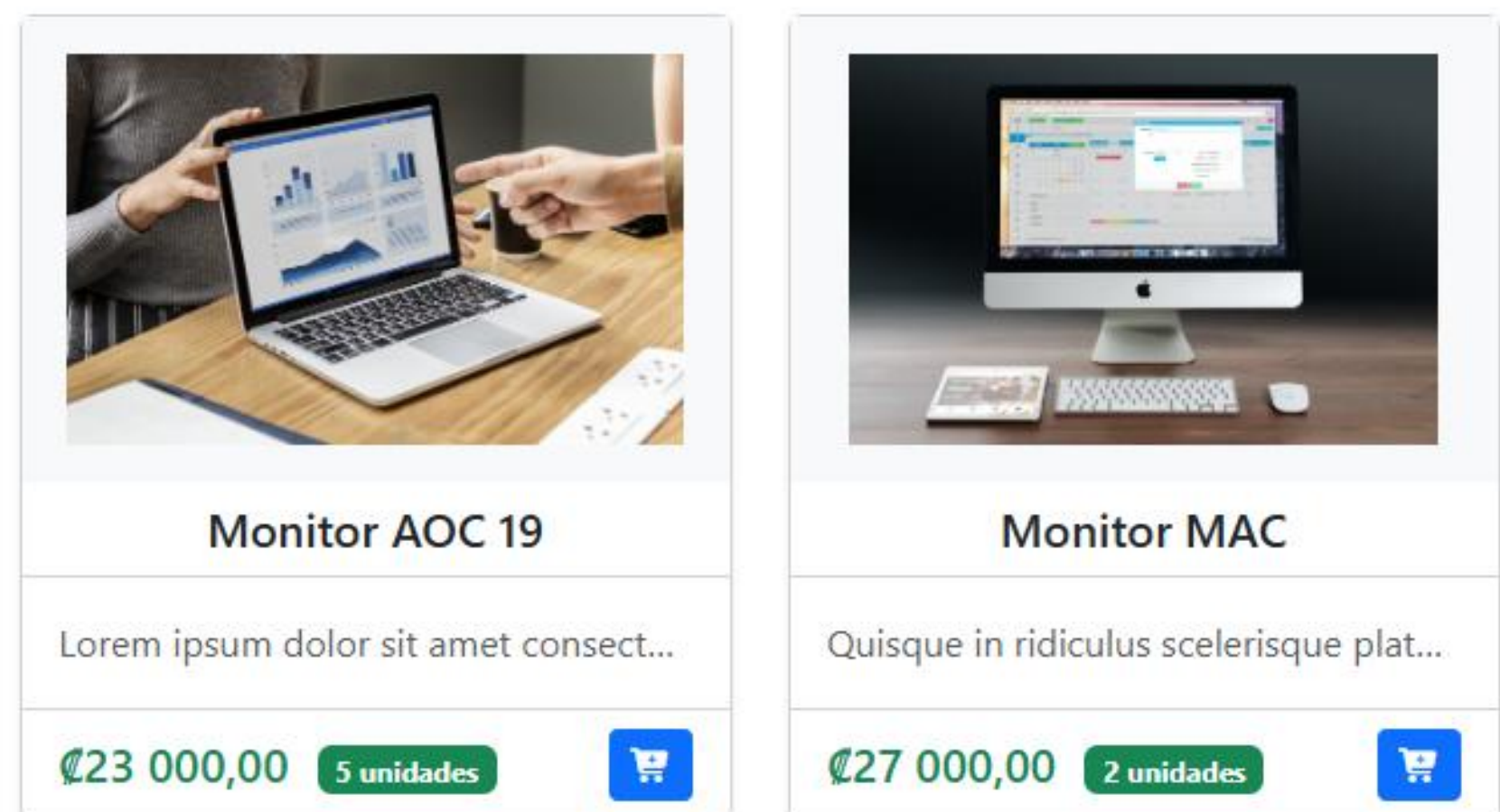
Restringiendo el menú con `sec:authorize`

La sentencia `sec:authorize` permite generar un código HTML si el resultado de su evaluación es verdadero, como por ejemplo:

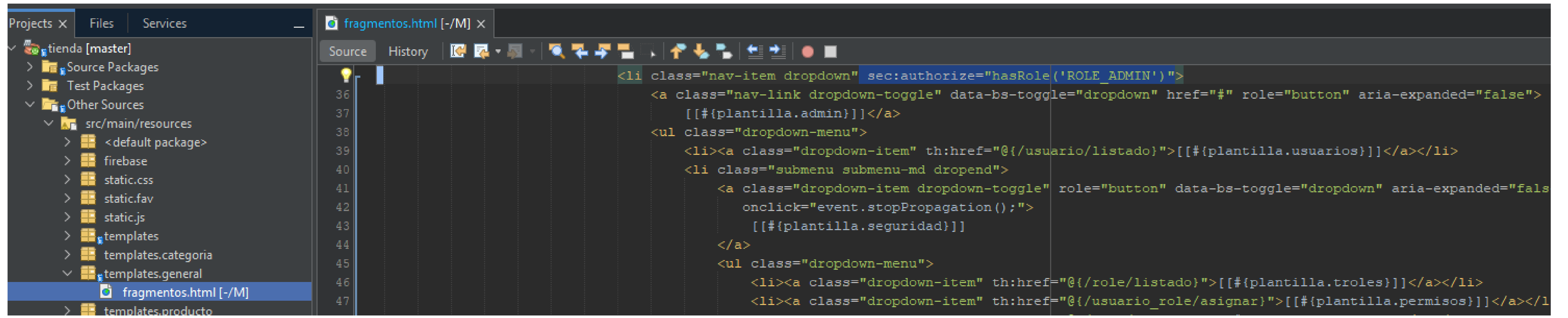
```
sec:authorize="hasRole('ROLE_VENDEDOR') or hasRole('ROLE_ADMIN')"
```



Solo quienes tienen perfil ADMIN o VENDEDOR.

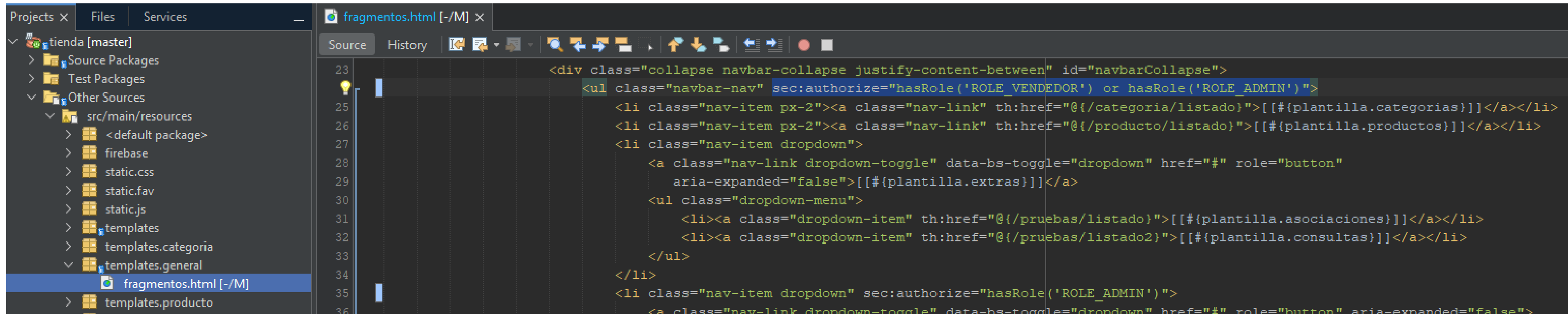


Solo ADMIN



```
fragmentos.html [-/M] x
Source History
36 <li class="nav-item dropdown" sec:authorize="hasRole('ROLE_ADMIN')">
37 <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button" aria-expanded="false">
38   [[#{plantilla.admin}]]</a>
39 <ul class="dropdown-menu">
40   <li><a class="dropdown-item" th:href="@{/usuario/listado}">[[#{plantilla.usuarios}]]</a></li>
41   <li class="submenu submenu-md dropend">
42     <a class="dropdown-item dropdown-toggle" role="button" data-bs-toggle="dropdown" aria-expanded="false"
43       onclick="event.stopPropagation();"
44       [[#{plantilla.seguridad}]]
45     </a>
46     <ul class="dropdown-menu">
47       <li><a class="dropdown-item" th:href="@{/role/listado}">[[#{plantilla.troles}]]</a></li>
48       <li><a class="dropdown-item" th:href="@{/usuario_role/asignar}">[[#{plantilla.permisos}]]</a></li>
```


Solo ADMIN o VENDEDOR



```
23 <div class="collapse navbar-collapse justify-content-between" id="navbarCollapse">
24     <ul class="navbar-nav" sec:authorize="hasRole('ROLE_VENDEDOR') or hasRole('ROLE_ADMIN')">
25         <li class="nav-item px-2"><a class="nav-link" th:href="@{/categoria/listado}">[[#{plantilla.categorias}]]</a></li>
26         <li class="nav-item px-2"><a class="nav-link" th:href="@{/producto/listado}">[[#{plantilla.productos}]]</a></li>
27         <li class="nav-item dropdown">
28             <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button"
29                 aria-expanded="false">[[#{plantilla.extras}]]</a>
30             <ul class="dropdown-menu">
31                 <li><a class="dropdown-item" th:href="@{/pruebas/listado}">[[#{plantilla.asociaciones}]]</a></li>
32                 <li><a class="dropdown-item" th:href="@{/pruebas/listado2}">[[#{plantilla.consultas}]]</a></li>
33             </ul>
34         </li>
35         <li class="nav-item dropdown" sec:authorize="hasRole('ROLE_ADMIN')">
36             <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button" aria-expanded="false">
```

Restringiendo la visualización de botones de acción con `sec:authorize`

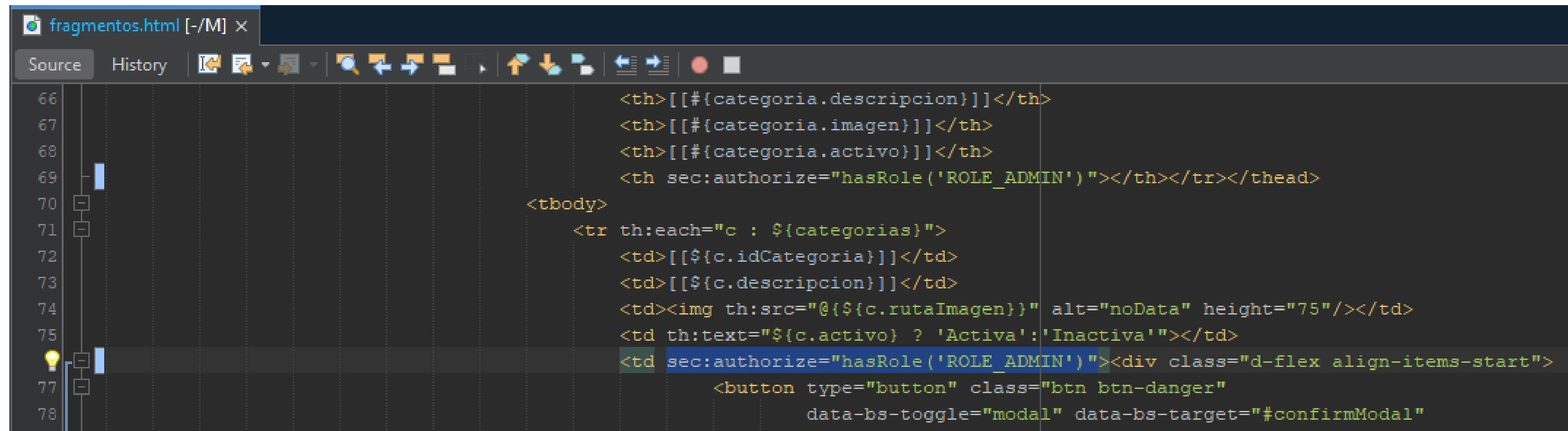
La sentencia `sec:authorize` permite generar un código HTML si el resultado de su evaluación es verdadero, como por ejemplo:

The screenshot shows a web application interface for managing categories. The top navigation bar includes links for TechShop, Categorías, Productos, Consultas, and Administración. A green banner reads "Su tienda en línea". Below this, a blue button labeled "+ Agregar Categoría" is highlighted with a purple box. The main content area displays a table titled "Listado de Categorías" with columns for #, Descripción, Imagen, and Activo. The table lists four categories: Monitores, Teclados, Tarjeta Madre, and Celulares, all marked as "Activa". To the right of the table, a green box shows "Total Categorías: 4". A purple box highlights the "Eliminar" (red button) and "Actualizar" (yellow button) actions for each category. A red box highlights a text annotation: "Estas acciones solo son permitidas por quien tiene perfil ADMIN.".

#	Descripción	Imagen	Activo
1	Monitores		Activa
2	Teclados		Activa
3	Tarjeta Madre		Activa
4	Celulares		Activa

`sec:authorize="hasRole('ROLE_ADMIN')"`

Limitando botones solo ADMIN (líneas 69 y 76)



```
66 <th>[[#{categoria.descripcion}]]</th>
67 <th>[[#{categoria.imagen}]]</th>
68 <th>[[#{categoria.activo}]]</th>
69 <th sec:authorize="hasRole('ROLE_ADMIN')"></th></tr></thead>
70 <tbody>
71 <tr th:each="c : ${categorias}">
72 <td>[[#{c.idCategoria}]]</td>
73 <td>[[#{c.descripcion}]]</td>
74 <td></td>
75 <td th:text="${c.activo} ? 'Activa': 'Inactiva'"></td>
76 <td sec:authorize="hasRole('ROLE_ADMIN')"><div class="d-flex align-items-start">
77 <button type="button" class="btn btn-danger"
78 data-bs-toggle="modal" data-bs-target="#confirmModal"
```


¿Cómo se ve?

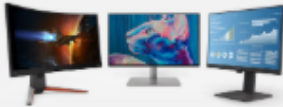
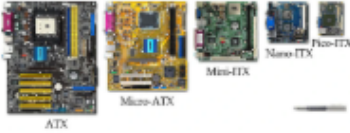
localhost/categoria/listado

Herramientas IAaprendizajeBiblioteca de Cursos

TechShopCategoríasProductosConsultasIdioma ▾rebeca Logout

Su tienda en línea

Listado de Categorías

#	Descripción	Imagen	Activo
1	Monitores		Activa
2	Teclados		Activa
3	Tarjeta Madre		Activa
4	Celulares		Activa

Total Categorías

4

Nos vemos en semana 10

RETO: Restrinja los botones en el listado producto también